

Apache Kafka

Partitions, Consumers & Scaling

Complete Revision Guide — Solutions Architect Edition

1. What Is a Kafka Partition?

A Kafka topic is not stored as one big file. It is split into partitions — ordered, immutable logs stored on disk. Each partition is the basic unit of parallelism, storage, and fault-tolerance in Kafka.

1.1 Core Mental Model

Think of a Kafka topic like a multi-lane highway:

- The topic is the highway.
- Each partition is one lane.
- Messages travel in strict order within a lane.
- Different lanes are fully independent.

NOTE

Messages within ONE partition are strictly ordered.

There is NO ordering guarantee across different partitions.

If you need global order for a key (e.g., all events for user-123), always route that key to the same partition using a consistent partition key.

1.2 Anatomy of a Partition

Component	What It Is	Key Point
Offset	Sequential integer per message in a partition	Immutable. Starts at 0.
Leader	The broker that handles all reads/writes	One leader per partition at any time.
Follower	Replicas that copy from the leader	Take over if leader fails.
ISR	In-Sync Replicas — followers caught up to leader	Only ISR members are eligible for leader election.
Log Segment	Physical file on disk that stores messages	Segments roll over by time or size.

2. Producers and Partition Assignment

When a producer sends a message, Kafka must decide which partition to write it to. The strategy depends on whether a partition key is provided.

2.1 Partition Assignment Rules

Scenario	Strategy	Result
No key provided	Round-robin (or sticky batching)	Messages spread across all partitions evenly.
Key provided	$\text{hash}(\text{key}) \% \text{numPartitions}$	Same key always lands on same partition.
Custom partitioner configured	Your logic	Full control over routing.

WARNING

Adding partitions to a topic AFTER creation changes the hash mapping.
A key that mapped to partition 3 before may map to partition 7 after.
This breaks ordering guarantees for existing keys. Plan partition count upfront — you cannot reduce it later.

2.2 Sticky Partitioner (Kafka 2.4+)

Before Kafka 2.4, keyless messages used round-robin per message, creating many small batches. The sticky partitioner batches keyless messages to one partition until the batch is full or `linger.ms` expires, then switches. This significantly reduces producer latency and broker load.

3. Consumers and Consumer Groups

Consumers read messages from Kafka. The Consumer Group model is how Kafka achieves parallel, scalable consumption without duplicating processing work.

3.1 The Golden Rule of Consumer Groups

CORE RULE

Each partition is assigned to exactly ONE consumer within a consumer group at any given time.
One consumer can own multiple partitions. But one partition cannot be shared by two consumers in the same group.
This guarantees ordered, non-duplicate processing per partition within a group.

3.2 Parallelism Limits

The number of partitions defines the maximum useful parallelism for a consumer group:

- Partitions = 6, Consumers = 3 → Each consumer handles 2 partitions. 

- Partitions = 6, Consumers = 6 → Each consumer handles 1 partition. Maximum parallelism. ✓
- Partitions = 6, Consumers = 8 → 2 consumers sit idle. ✗ Wasted resources.

NOTE

Adding more consumers than partitions does NOT increase throughput.
The extra consumers will remain idle until a rebalance occurs (e.g., if an active consumer dies).
They can serve as hot standby consumers for fast failover.

3.3 Multiple Consumer Groups

Different consumer groups are fully independent. Each group maintains its own offset pointer per partition. This means the same topic can be consumed independently by multiple applications without interfering with each other.

- Group A: analytics service — reads from offset 0
- Group B: email notification service — reads from offset 0 independently
- Group C: audit log service — reads from its own offset

✓ TIP

This is the pub-sub model in Kafka.
Each consumer group = an independent subscriber to the topic.
Contrast with the queue model: one group, multiple consumers sharing the workload.

4. Replication and Fault Tolerance

4.1 Replication Factor

Every partition can be replicated across multiple brokers. The replication factor (RF) controls how many copies exist.

Replication Factor	Meaning	Fault Tolerance
RF = 1	No redundancy. Single copy.	Zero. One broker fails = data lost.
RF = 2	Leader + 1 follower.	Survives 1 broker failure.
RF = 3	Leader + 2 followers. Production default.	Survives 2 broker failures.

4.2 Leader Election and ISR

When a partition leader dies, Kafka elects a new leader from the ISR (In-Sync Replicas). An ISR member is a follower that is caught up to the leader within `replica.lag.time.max.ms` (default: 30 seconds).

- Only ISR members are eligible for leader election.
- If ISR shrinks to just the leader and the leader dies, Kafka must wait for a replica to catch up, or elect an out-of-sync replica (risk of data loss).

- `min.insync.replicas` (`min.isr`) controls the minimum ISR size before the leader refuses writes.

NOTE

Production recommended settings:

`replication.factor = 3`

`min.insync.replicas = 2`

`acks = all` (producer waits for all ISR to acknowledge)

This combination ensures no data loss even if one broker dies.

5. Offset Management

An offset is the position of a message within a partition. Offsets are how Kafka tracks what each consumer group has processed.

5.1 Commit Strategies

Strategy	How It Works	Risk	Use Case
Auto-commit	Kafka commits offsets every <code>auto.commit.interval.ms</code>	At-least-once: may reprocess on crash	Simple apps, idempotent processing
Manual sync commit	Consumer calls <code>commitSync()</code> after processing	Blocking. Lower throughput.	Critical data, exactly-once semantics
Manual async commit	Consumer calls <code>commitAsync()</code> . Non-blocking.	May skip failed commits.	High-throughput pipelines

5.2 Delivery Semantics

- At-most-once: commit before processing. Message lost if crash after commit.
- At-least-once: commit after processing. Message reprocessed if crash before commit. Most common.
- Exactly-once: requires Kafka transactions + idempotent producers. Highest complexity.

5.3 `__consumer_offsets` Topic

Kafka stores committed offsets in an internal topic called `__consumer_offsets`. This replaced ZooKeeper-based offset storage in Kafka 0.9. It is compacted (retains only the latest offset per group-topic-partition key) and has 50 partitions by default.

6. Rebalancing

A rebalance is the process of reassigning partition ownership among consumers in a group. It is triggered by:

- A consumer joining the group

- A consumer leaving or crashing
- A new partition being added to a subscribed topic
- A consumer calling unsubscribe()

WARNING

During a rebalance, ALL consumers in the group stop processing (Stop-The-World).
For large consumer groups or slow rebalancers, this can cause visible lag spikes.
Frequent rebalances are a common source of consumer group instability.

6.1 Rebalance Strategies

Strategy	Behaviour	Best For
Range (default)	Assigns contiguous partitions to consumers.	Simple setups, single-topic subscriptions.
RoundRobin	Distributes partitions evenly across consumers.	Multi-topic subscriptions, balanced load.
Sticky (recommended)	Minimises partition movement during rebalance.	Reducing rebalance impact in production.
Cooperative Sticky	Incremental rebalance — no full stop-the-world.	Kafka 2.4+, low-latency consumer groups.

TIP

In production, prefer `partition.assignment.strategy = CooperativeStickyAssignor`.
It performs incremental rebalances — only the partitions that need to move are revoked, instead of revoking all partitions from all consumers simultaneously.

7. Scaling Kafka

7.1 Scaling Producers

Producers scale independently of brokers and partitions. Key tuning levers:

Config	Default	Tune For
<code>batch.size</code>	16 KB	Higher value → larger batches → better throughput
<code>linger.ms</code>	0 ms	Add delay to allow batch to fill up before sending
<code>buffer.memory</code>	32 MB	In-memory buffer before producer blocks
<code>compression.type</code>	none	lz4 or snappy for throughput; zstd for compression ratio
<code>max.in.flight.requests.per.connection</code>	5	Set to 1 for strict ordering with retries

7.2 Scaling Consumers

Consumer throughput is bounded by the number of partitions. To scale consumer throughput:

1. Increase partition count (must be done upfront or carefully after creation).
2. Add more consumer instances up to the partition count limit.
3. Tune `max.poll.records` — how many messages fetched per poll loop.
4. Use multiple threads per consumer (advanced — requires careful offset management).

7.3 Scaling Brokers

Kafka scales horizontally by adding brokers. Partition leadership is distributed across brokers. Adding brokers does not automatically rebalance existing partitions — you must run `kafka-reassign-partitions.sh` or use Cruise Control to redistribute.

- More brokers = more disk, network, and CPU for partition replicas.
- Each broker should ideally be leader for a roughly equal number of partitions (preferred replica leader election).
- Keep partition count per broker under 4,000 and total partition count per cluster under 200,000 (practical limits based on ZooKeeper/KRaft metadata load).

8. How Many Partitions Should You Create?

This is one of the most common and most misunderstood Kafka design questions. There is no universal answer, but here is a structured decision framework:

8.1 Decision Framework

Factor	Guidance
Target throughput	$\text{Partitions} = \max(\text{target_MB/s} \div \text{producer_MB/s_per_partition}, \text{target_MB/s} \div \text{consumer_MB/s_per_partition})$
Consumer parallelism	At least as many partitions as your maximum planned consumer count.
Retention and disk	More partitions = more open file handles and more disk fragmentation.
Replication overhead	Each partition replica consumes network bandwidth during replication.
Rebalance speed	More partitions = longer rebalance time. Balance with need for parallelism.

8.2 General Rules of Thumb

- Start with partitions = number of consumer instances you plan to run at peak.
- For low-throughput topics: 3–6 partitions is often sufficient.
- For high-throughput topics (100 MB/s+): 30–100 partitions is common.
- Avoid creating thousands of partitions per topic unless throughput demands it.

WARNING

Over-partitioning is a real problem.

Each partition = memory, file handles, and network connections on every broker.

A cluster with 100 topics × 100 partitions × 3 replicas = 30,000 partition replicas.

This strains ZooKeeper (or KRaft) and slows controller failover.

9. Common Misconceptions Corrected

Misconception	Reality
More consumers always = more throughput	False. If consumers > partitions, excess consumers are idle. Throughput is bounded by partition count.
Adding partitions is always safe	False. It breaks key-based partitioning for existing data and cannot be undone (no partition reduction).
Kafka guarantees global message order	False. Order is guaranteed ONLY within a single partition. Cross-partition order is not guaranteed.
Replication slows down producers significantly	Partially true. With acks=all and min.isr=2, RF=3 adds latency. But Kafka's async replication is designed to minimise this.
Consumer lag always means slow consumers	False. Lag spikes also occur during rebalances, GC pauses, or when partition leadership moves.
Kafka deletes messages after consumption	False. Kafka retains messages based on retention.ms or retention.bytes, regardless of consumer offset position.

10. Solutions Architect Quick Reference

10.1 When to Choose What

Requirement	Kafka Design Choice
Strict ordering per entity (e.g., per user)	Use entity ID as partition key. Consistent hashing guarantees same partition.
Maximum consumer parallelism	Partitions ≥ max consumer count. Use CooperativeStickyAssignor.
Zero data loss	RF=3, min.isr=2, acks=all, enable.idempotence=true.
Highest throughput (acceptable data loss risk)	acks=1 or acks=0, larger batches, compression=lz4.
Multiple independent apps reading same topic	Separate consumer group per application.
Reprocessing old messages	Reset consumer group offset with <code>kafka-consumer-groups --reset-offsets</code> .
Exactly-once end-to-end	Kafka Transactions API + idempotent producer + transactional consumer.

10.2 Key Metrics to Monitor

- `consumer_lag` (records behind latest offset) — primary health indicator.
- `under_replicated_partitions` — should always be 0. Non-zero = broker or network issue.
- `active_controller_count` — should always be exactly 1. Zero = no controller. Two = split-brain.
- `request_handler_avg_idle_percent` — below 20% means broker is saturated.
- `network_processor_avg_idle_percent` — below 30% means network threads are bottlenecked.

TL;DR — One-Page Cheat Sheet

Concept	Key Fact
Partition	Basic unit of parallelism. Ordered log. Cannot be reduced after creation.
Consumer Group Rule	1 partition → 1 consumer (per group). Consumers > partitions = idle consumers.
Ordering	Guaranteed within partition. NOT across partitions.
Replication Factor	Use RF=3 + min.isr=2 in production. Never RF=1 for important topics.
Offset	Message position in partition. Consumers track their own offset per partition.
Rebalance	Triggers when group membership changes. Use CooperativeStickyAssignor to minimise impact.
Partition key	Determines partition. Same key → same partition. Plan before creating topics.
Scaling consumers	Add consumers up to partition count. Beyond that, no gain.
Scaling throughput	More partitions + more consumers + tuned batch size + compression.
Consumer lag	Records not yet consumed. Monitor continuously. Spikes indicate rebalance or slow consumers.

End of Document